R. P. Shaha University

- an institution of Kumudini Welfare Trust of Bengal (BD) Ltd.

VEXY THE VERTEX

An Animated Educational Cartoon for the Graphics Pipeline

Course Title	Computer Graphics Lab
Course Code	0613-CSE-4013
Submitted By	MD. Osiur Rahman Rahif ID: 23300149
Submitted To	Tanjina Akter Assistant Professor, CSE, RPSU
Submission Date	11/11/2026

1. Introduction, Objectives and Scope

1.1 Introduction

Computer graphics converts geometric descriptions and visual attributes into images. Concepts such as vertices, transformations, interpolation, rasterization, depth testing, lighting, and projection can be difficult to understand from equations alone. Vexy the Vertex addresses this learning challenge with a short animated sequence that presents each concept visually and places relevant OpenGL code beside the animation.

1.2 Objectives

- Introduce vertices and primitive assembly through the construction of a triangle.
- Demonstrate translation, rotation, scaling, and the effect of transformation order.
- Compare flat shading with smooth color interpolation across a triangle.
- Explain pixel coverage using a grid and three edge-function tests.
- Show the roles of depth testing, surface normals, lighting, and projection in a 3D scene.
- Summarize the lessons as a staged graphics-pipeline animation.
- Support learning with captions, code snippets, navigation, pause, and progress feedback.

1.3 Scope

The application is a teaching demonstration, not a general-purpose renderer or a full graphics-engine implementation. It uses a classic OpenGL fixed-function context to keep the API calls visible and direct. It includes a custom 2D rasterization demonstration for a triangle grid and a real OpenGL 3D scene for depth, lighting, and projection comparisons.

2. Technology Stack

Technology	Role in the application
Python	Application language; coordinates scene timing, animation calculations, and event flow.
Pygame	Creates the desktop window, reads keyboard and quit events, provides frame timing, and rasterizes font glyphs.
PyOpenGL: OpenGL.GL	Provides immediate-mode geometry, color, matrix, viewport, texture, depth, and lighting operations.
PyOpenGL: OpenGL.GLU	Provides GLU utilities including perspective setup, camera positioning, quadric creation, and sphere rendering.
Python standard library	math supports geometry and easing; random creates repeatable finale confetti; sys provides the process exit path.

3. Application Architecture

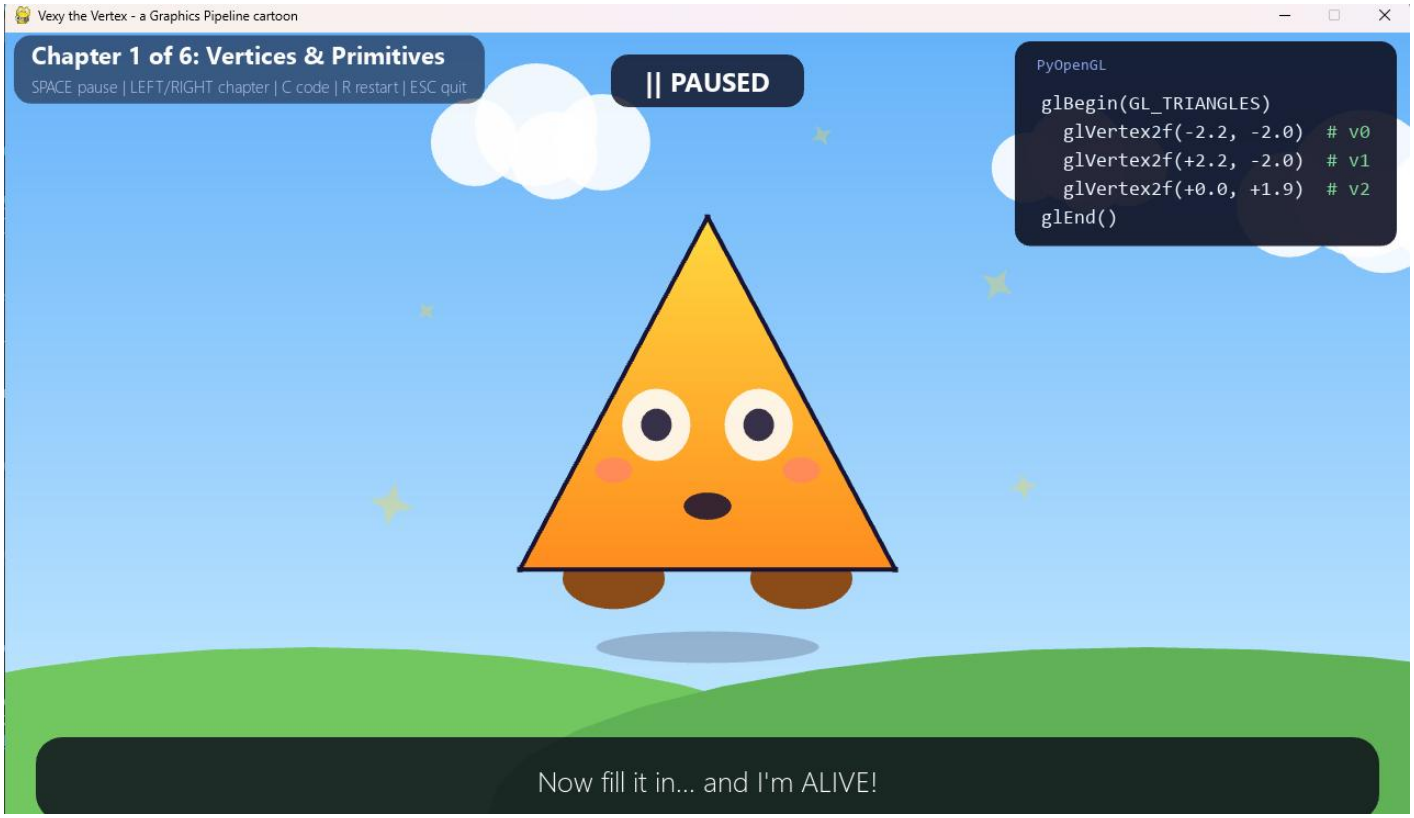
The implementation is contained in one Python module but is divided into logical layers. The top-level application initializes the graphics window and runs the event/update/render loop. The Player class manages scene selection and time. Each lesson is represented by a class derived from Scene. Reusable drawing functions and TextCache keep rendering code consistent.

3.1 Main Components

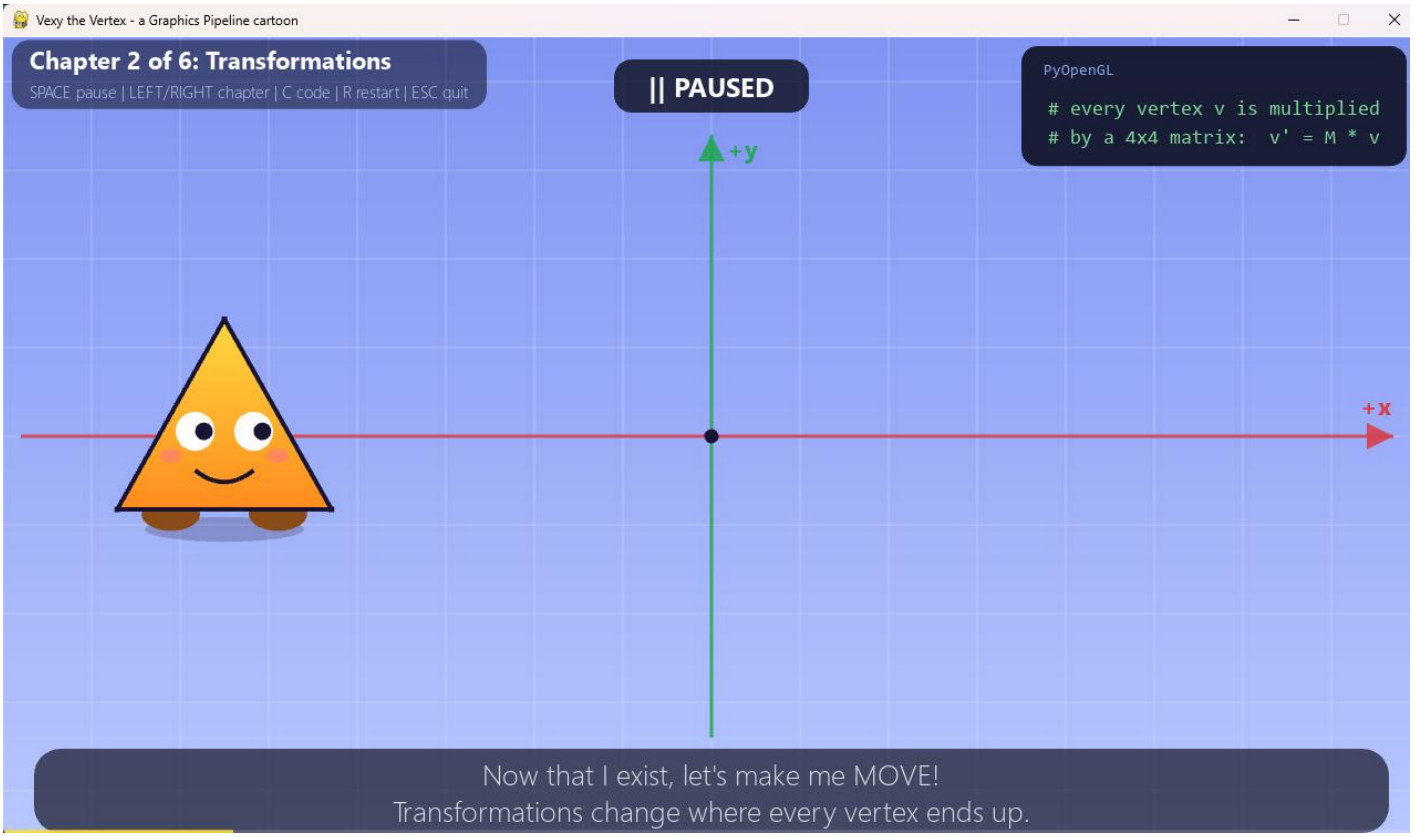
Component	Responsibility
Window and runtime setup	init_window() initializes Pygame, requests a 24-bit depth buffer, attempts multisampling, and creates the OpenGL window.
Scene base class	Defines shared metadata, caption lookup, drawing interface, code-panel interface, and character talk timing.
Chapter classes	Implement the scene-specific animation, explanatory captions, and time-dependent code snippets.
Player	Owns the scene list, current index, local scene time, pause state, code-panel visibility, progress calculation, and drawing sequence.
Drawing helpers	Render rectangles, ellipses, circles, triangles, lines, outlines, arrows, grids, gradients, and rounded panels.
TextCache	Caches rendered font textures so repeated text can be drawn as OpenGL textured quads.

4. Scene-by-Scene Functional Description

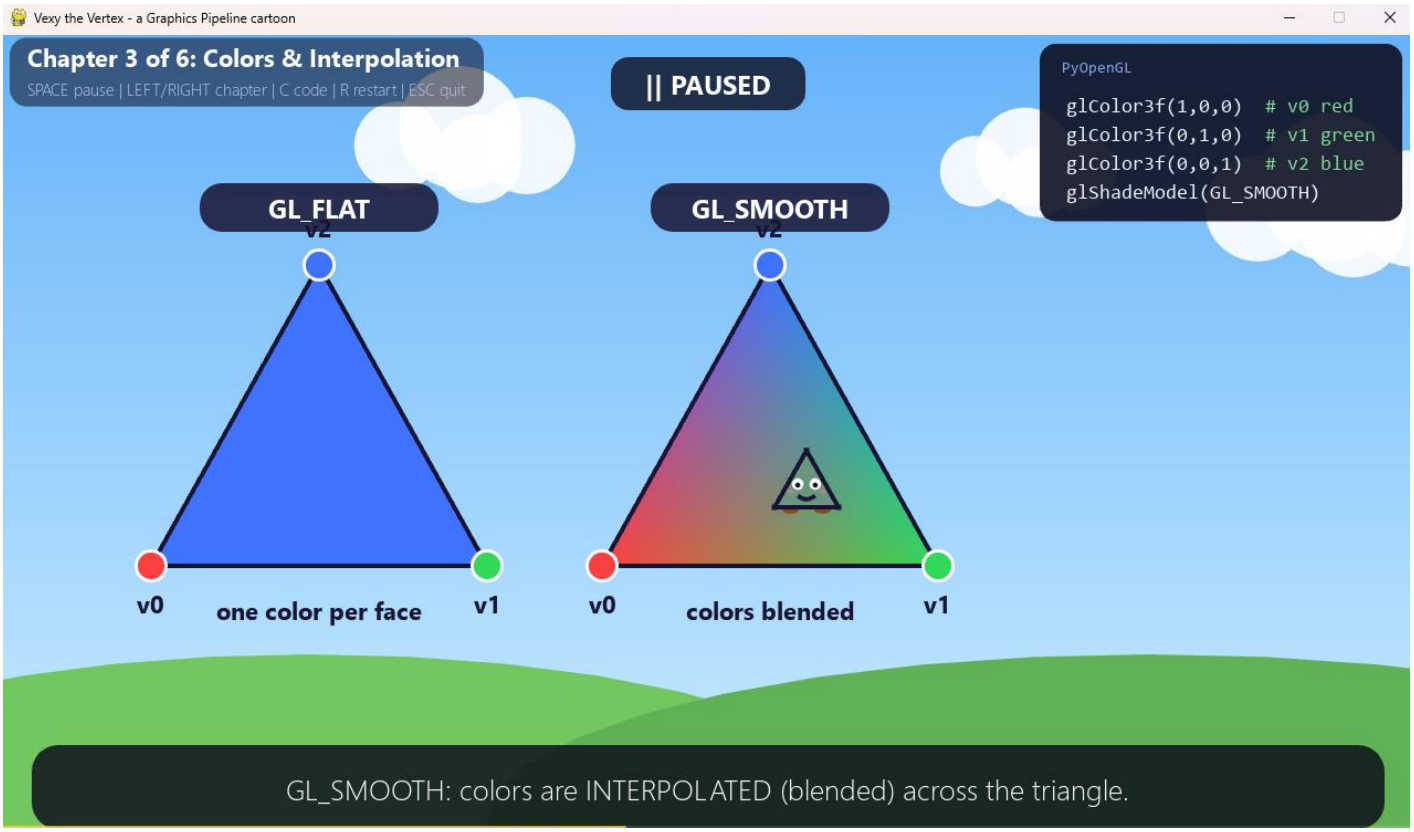
4.1 Chapter 1 – Vertices and Primitives



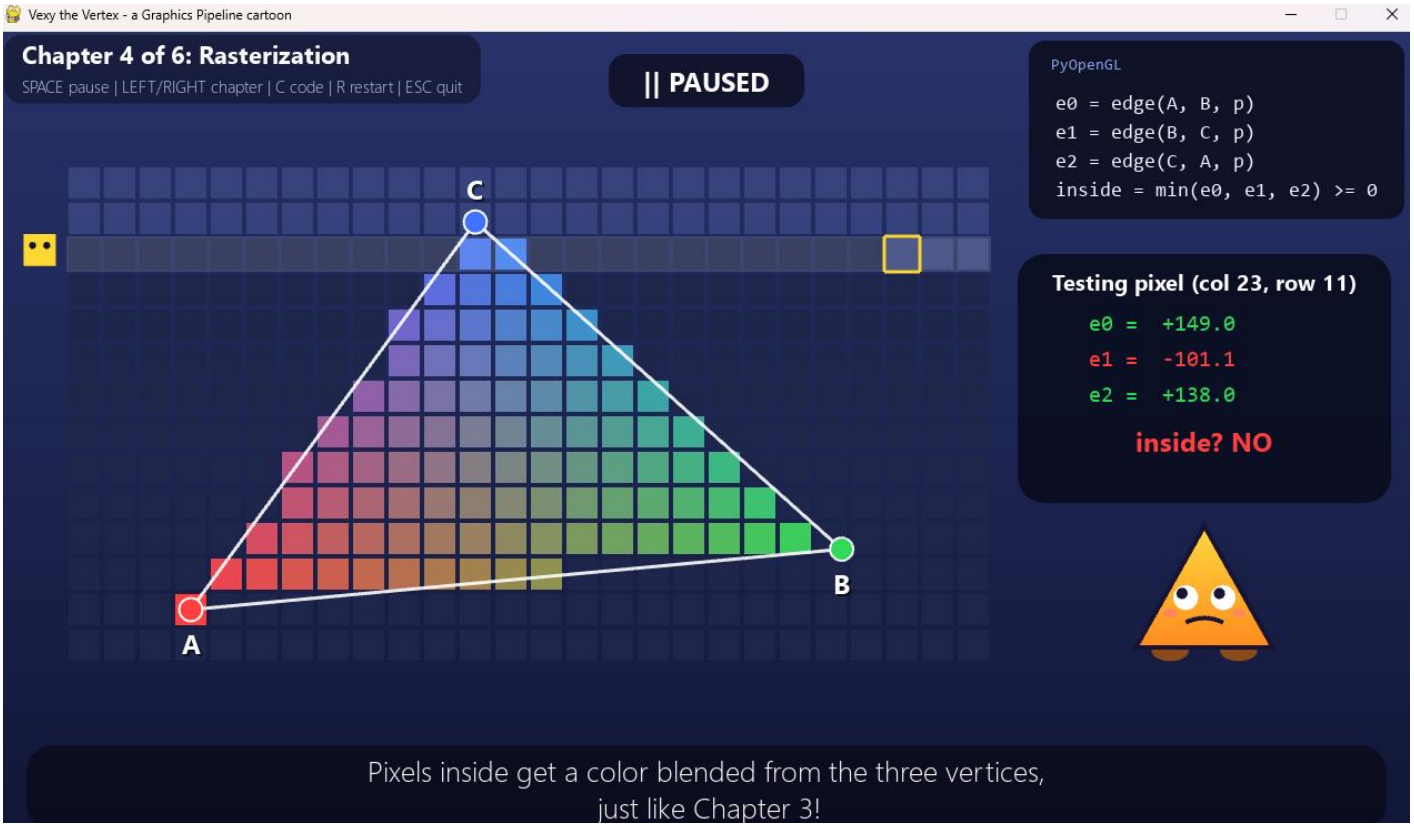
4.2 Chapter 2 – Transformations



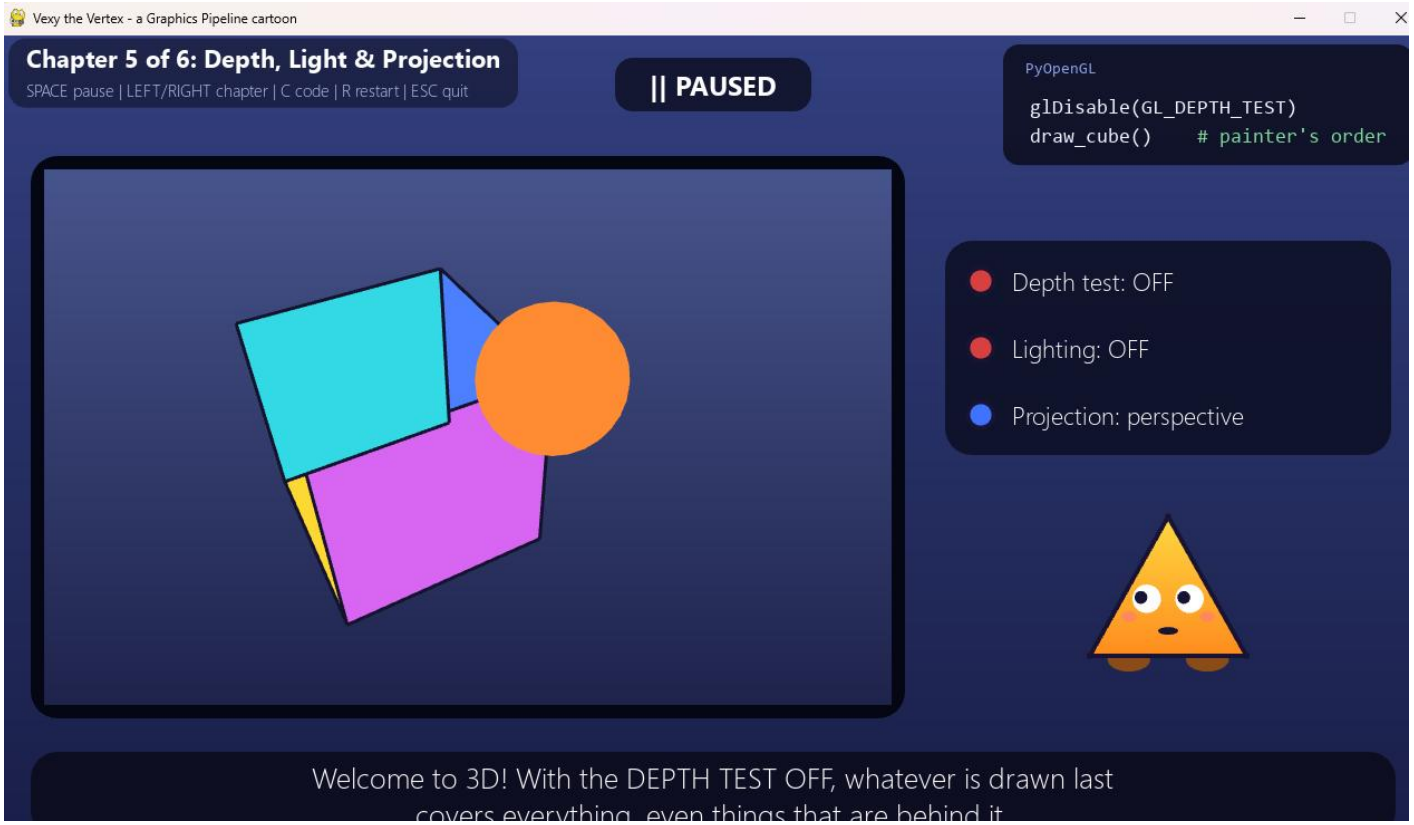
4.3 Chapter 3 – Colors and Interpolation



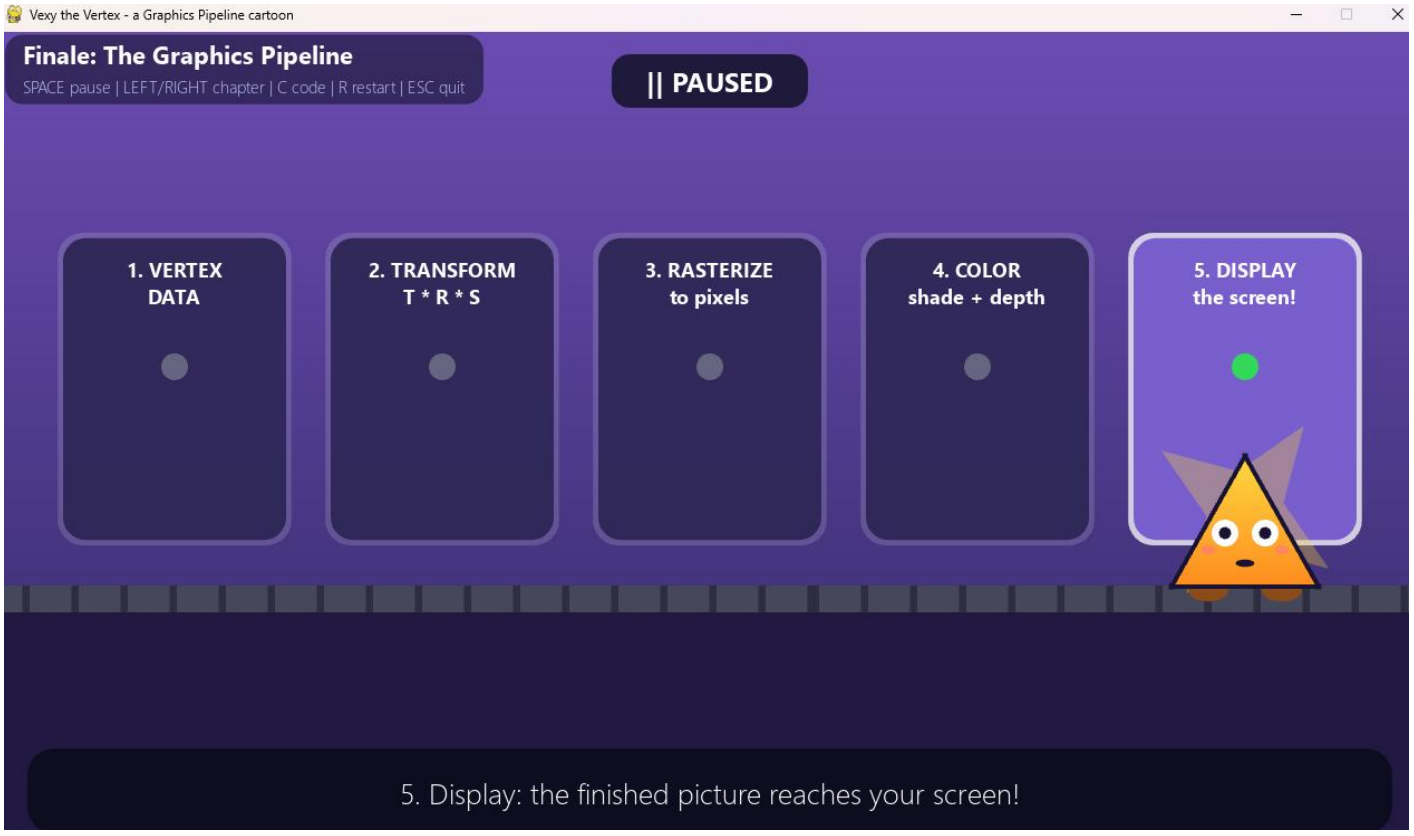
4.4 Chapter 4 – Rasterization



4.5 Chapter 5 – Depth, Light and Projection



4.6 Finale – The Graphics Pipeline



5. Core Algorithms and Mathematical Concepts

5.1 Geometric Transformations

The transformation lesson applies standard 2D operations to the local coordinates of Vexy. Translation moves every vertex by the same offset. Rotation changes each vertex direction around the origin. Scaling changes each vertex distance from the origin.

```
Translation:  x′ = x + dx ,   y′ = y + dy
Rotation:  x′ = x cos(α) − y sin(α) ,   y′ = x sin(α) + y cos(α)
Scaling:  x′ = sx · x ,   y′ = sy · y
```

The implementation uses OpenGL matrix-stack calls and push/pop operations to isolate local transformations. The order comparison is deliberately visual: changing the order of translation and rotation changes whether the character spins at a fixed offset or orbits the origin.

5.2 Interpolated Color

The color chapter combines the RGB values stored at the three triangle vertices. For weights w0, w1, and w2, the interpolated color is: C(p) = w0 · C0 + w1 · C1 + w2 · C2. The sample weights are normalized so that they add to one. The same conceptual weighting is used in the rasterization scene to calculate the displayed color of a covered cell.

5.3 Edge Function and Pixel Coverage

The edge helper computes a signed 2D cross-product value for a directed edge and a test point: edge(a, b, p) = (b.x − a.x) · (p.y − a.y) − (b.y − a.y) · (p.x − a.x). The sign indicates which side of the directed edge contains the point. The implementation tests the center of each grid cell, checks all three triangle edges, and stores the classification and edge values before the animated scan begins. This precomputation allows the scan animation to reveal the results incrementally.

5.4 Animation Timing and Easing

Helper	Purpose in the source
clamp(x, lo, hi)	Restricts a value to a specified interval, commonly 0 to 1.
lerp(a, b, k)	Linearly interpolates between two values.
seg(t, a, b)	Calculates normalized progress through a time interval.
smoothstep(k)	Produces a smooth start and stop for transitions.
ease_out_back(k)	Adds a small overshoot before an animated value settles.
mix(c0, c1, k)	Interpolates between two RGB colors.

6. Installation and Execution

6.1 Prerequisites

- A Python 3 environment compatible with the installed Pygame and PyOpenGL packages.
- Pygame and PyOpenGL installed in the same Python environment used to run the script.
- A desktop session and graphics driver that provide an OpenGL context compatible with the fixed-function API calls used by the project.

6.2 Install Dependencies

Open a terminal in the project directory and install the two external Python packages used by the source:

```
python -m pip install pygame PyOpenGL
```

6.3 Runtime Configuration

Setting	Configured value / behavior
Window size	1280 × 720 pixels.
Target frame rate	60 frames per second; frame delta is capped at 0.1 seconds.
2D coordinate space	x from −8 to +8; y from −4.5 to +4.5.
Depth buffer request	24-bit depth buffer.
Multisampling	Attempts one multisample buffer with four samples; retries without multisampling if setup fails.
3D stage viewport	(40, 120, 760, 480) in window pixel coordinates.

7. Verification and Quality Considerations

7.1 Verification Performed for This Documentation

The supplied Python source was checked with Python bytecode compilation (py_compile) during preparation of this document. The syntax check completed successfully. The interactive application was not launched in a live desktop/OpenGL session as part of this documentation task, so visual runtime behavior, frame rate, and hardware compatibility have not been independently confirmed here.

Verification item	Evidence / status
Python syntax compilation	PASS – source compiled with py_compile.
Scene sequence and controls	Reviewed from the Player and main-loop implementation; not separately runtime-tested.
2D and 3D rendering output	Not runtime-tested during documentation preparation.
OpenGL compatibility across devices	Not tested; depends on the target system and graphics driver.
Automated test suite	No dedicated automated test suite is included in the supplied source.

7.2 Suggested Manual Test Plan

Test	Procedure	Expected observation
Startup	Run the script from a graphical desktop session.	A 1280 × 720 window opens and the first scene begins.
Pause/resume	Press SPACE during playback.	The current scene time stops and resumes.
Scene navigation	Use LEFT and RIGHT at different points in a scene.	The scene changes or restarts according to the implemented timing rule.
Code panel	Press C.	The live-code panel visibility toggles.
Restart	Press R.	Playback returns to the first scene.
3D demonstration	Observe Chapter 5 through all timed phases.	Depth, lighting, perspective, and orthographic states are presented in sequence.
Exit	Press ESC or close the window.	The main loop exits and Pygame is shut down.

8. Current Limitations

- The project uses legacy fixed-function OpenGL and immediate mode. These APIs are intentionally simple for instruction but are not the recommended foundation for new high-performance graphics applications.
- The window dimensions, world coordinates, and several scene-specific layouts are configured as constants in the source rather than exposed as user settings.

- The rasterization lesson uses a custom pixel-center edge test for one triangle and a fixed grid. It is an educational demonstration, not a complete implementation of every hardware rasterization rule.
- The program is delivered as one Python file. There is no separate configuration file, test package, installer, or user guide bundled with the source.
- No audio narration, persistent user progress, or interactive editing interface is implemented; the learner experiments by modifying the source code.

9. Future Development Opportunities

The source includes a set of suggested experiments. The following extensions remain consistent with the project's current teaching goals:

Area	Possible extension
Geometry	Change Vexy's BODY vertices; add a mirrored transformation using negative scale; compare triangle winding.
Primitive types	Extend the color lesson with GL_QUADS or GL_TRIANGLE_FAN.
Rasterization	Add a bounding-box optimization, an alternative barycentric inside test, or multisample coverage for a simple anti-aliasing comparison.
3D visibility	Enable face culling and demonstrate how front-face winding affects visible surfaces.
Modern OpenGL	Port a chapter to vertex buffer objects (VBOs), vertex array objects (VAOs), and GLSL vertex/fragment shaders.
Texture mapping	Add a new chapter that assigns texture coordinates and renders a checkerboard texture.
Project engineering	Separate the single source file into focused modules and add automated tests for mathematical helpers, scene timing, and edge-function classification.

10. Learning Outcomes

- Represent a triangle using vertex positions and explain translation, rotation, scaling, and transformation order.
- Explain flat and smooth shading, barycentric color interpolation, and pixel coverage using edge tests.
- Describe depth buffering, surface normals, lighting, and perspective versus orthographic projection.
- Distinguish Pygame application management from OpenGL rendering and identify a path toward shader-based graphics.

11. Conclusion

Vexy the Vertex introduces foundational computer-graphics concepts through six connected animated scenes. Its live-code panel and keyboard controls support guided learning and independent experimentation. The fixed-function OpenGL implementation is an educational starting point, with a clear route toward shader-based rendering and further graphics topics.